SPACEMAN PROJECTS

# Driver App Architecture
# Courier Operations Blueprint

Detailed mobile UI/UX, assignment, pickup, Google Maps routing, location sharing, delivery proof, exceptions, security, battery reliability and monorepo implementation flow.

DRIVER APP | ADMIN WEB | MERCHANT | CUSTOMER | MAPS

## Architecture principles

1. **Assignments are explicit and race-safe** — A driver cannot claim an order already assigned to someone else.
2. **Location is purpose-limited** — Tracking is collected only for active delivery operations under a clear policy.
3. **Status changes follow a matrix** — Accept, pickup, on-the-way and delivered commands are validated server-side.
4. **Maps enrich; they do not own orders** — Route and ETA data are normalized into the canonical delivery record.
5. **Mobile reliability matters** — Resume, connectivity, battery and background behavior are designed into the workflow.

## DOCUMENT SCOPE

1. **Role experience** — Screens, commands, states and UX
2. **Workflow ownership** — Who reads, writes, approves and resolves
3. **Data contracts** — Canonical records, snapshots and enums
4. **Security boundaries** — Rules, Functions, App Check and audit
5. **Monorepo flow** — Shared packages without forced UI reuse
6. **Development plan** — Phases, tests, deployment and Codex prompts

VERSION 1.0 - JULY 2026

Prepared as a stable visual reference for Codex prompts and implementation reviews.

SYSTEM CONTEXT

# Driver App Role in the Five-App System

The Driver App executes assigned delivery work and publishes controlled location/status updates to the shared order record.

## 1 Driver capabilities

1. **Workspace** — Available and assigned orders
2. **Assignment** — Accept / acknowledge work
3. **Navigation** — Pickup and drop-off routes
4. **Location** — Share active-delivery position
5. **Delivery** — On-way, proof and delivered

## 2 Shared domain layer

1. **types/** — Order, assignment, location
2. **schemas/** — Driver command payloads
3. **services/** — Assigned orders and status
4. **maps/** — Route and coordinate types
5. **notifications/** — Assignment event mapping

## 3 Backend services

1. **Auth** — Driver identity and status
2. **Firestore** — Assignments and order truth
3. **Functions** — Race-safe assignment/status
4. **FCM** — New work and order updates
5. **Storage** — Optional proof-of-delivery media

## 4 Cross-app effects

1. **Admin** — Assign/reassign and monitor
2. **Merchant** — Pickup readiness / arrival
3. **Customer** — Safe status and tracking
4. **Maps** — Route, ETA and navigation
5. **Activities** — Stale/offline/exception alerts

**Operational boundary**

Driver App reads assigned orders and submits delivery commands. It cannot mark payment paid, edit catalog/store data, refund, modify another driver's assignment or expose unrestricted customer information.

# Driver Mobile Shell and Operational Components

A glanceable, one-handed interface prioritizes current work, navigation and safe status changes.

## 1 Navigation and workspace

### 1 Orders workspace

Available jobs when policy permits, assigned queue and active delivery.

### 2 Notifications

Assignment, merchant readiness, admin changes and exception alerts.

### 3 Account / availability

Driver identity, online state, permissions and sign out.

### 4 Active order focus

One dominant action per current lifecycle state.

## 2 Reusable mobile components

### 1 Delivery card

Order ID, store, address summary, status, distance and priority.

### 2 Status stepper

Assigned, pickup, on the way and delivered with explicit ownership.

### 3 Map panel

Lazy-loaded route, location freshness and external navigation action.

### 4 Safety confirmation

Confirm pickup/delivery; prevent accidental tap and repeated command.

### 5 Offline state

Connection banner, queued location policy and canonical refresh on resume.

### 6 Accessibility

Large targets, readable contrast, screen-reader labels and reduced motion.

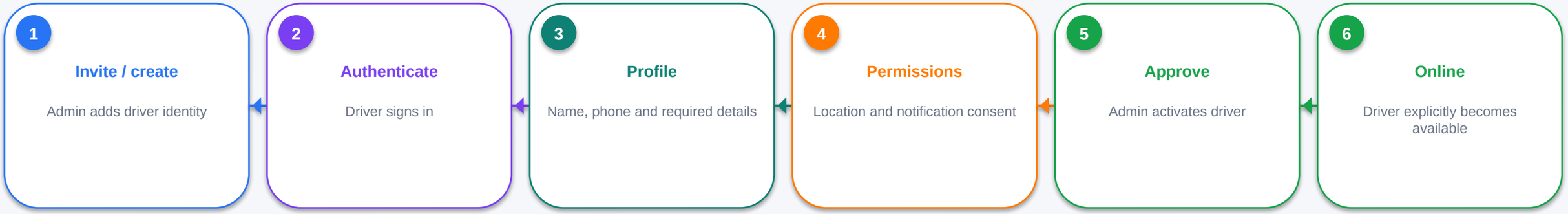
**Driving safety UX**

The app should minimize text entry and complex interaction while moving. Navigation handoff and status commands must not encourage unsafe device use.

# Driver Identity, Approval and Availability

Drivers enter the operational pool only after an admin-controlled role and onboarding state are valid.

## 1 Onboarding flow




## 2 Driver state model

| State | Can sign in | Available for assignment | Can read assigned order | Admin control |
|---|---|---|---|---|
| invited | Limited | No | No | Resend / revoke |
| pending_profile | Yes | No | No | Review missing data |
| pending_approval | Yes | No | Optional onboarding only | Approve / reject |
| active_offline | Yes | No | Existing assigned work by policy | Suspend / assign |
| active_online | Yes | Yes | Yes | Assign / monitor |
| suspended | Blocked | No | No operational access | Unsuspend |
| archived | No | No | Historical references retained | Retention workflow |

**Availability is not authentication**

A valid driver account may be offline and unavailable. Online state, freshness and current assignment are operational fields separate from identity.

# Available, Assigned and Active Delivery Views

The workspace shows only actionable work and avoids mixing unrelated operational states.

## 1 Available orders

### 1 Eligibility filter

Paid, dispatchable, in zone and driver meets policy.

### 2 Limited details

Enough pickup/drop-off context without exposing unnecessary personal data.

### 3 Accept action

Race-safe backend command; no client-only assignment write.

### 4 Empty state

No jobs available, offline, suspended or permission issue are distinct.

## 2 Assigned queue

### 1 Priority ordering

Active delivery first, then pickup readiness / assigned time.

### 2 Merchant context

Store name, pickup address, order readiness and instructions.

### 3 Customer context

Delivery address snapshot and permitted instructions.

### 4 Admin changes

Reassignment or cancellation refreshes immediately.

## 3 Active delivery

### 1 Single current action

Navigate to store, confirm pickup, navigate to customer or deliver.

### 2 Live freshness

Last location update, route timestamp and connection state.

### 3 Issue entry

Address, merchant, vehicle, safety or customer problem.

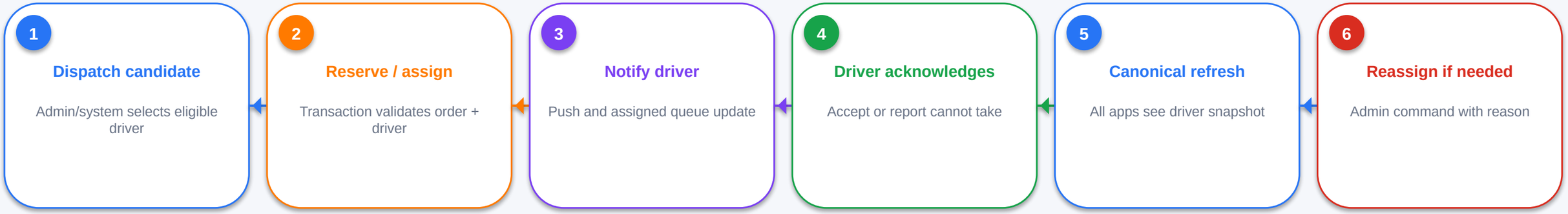
### 4 Completion

Proof policy, deliveredAt and canonical confirmation.

# Assignment, Acceptance and Race Prevention

Dispatch may be admin-led initially, but every assignment mutation remains atomic and auditable.

## 1 Assignment flow




## 2 Assignment invariants

| Invariant | Validation | Failure result |
|---|---|---|
| **One active driver per order** | driverId absent or expected version matches | Conflict; refresh order |
| **Driver eligible** | active, online/allowed, not suspended, zone/capacity policy | Reject assignment |
| **Order eligible** | paid and in allowed dispatch states | Reject assignment |
| **Stale accept denied** | assignment version / driverId changed | No overwrite |
| **Reassignment audited** | Reason, previous driver, new driver and actor | Activity on failure |
| **Notification idempotent** | Event key based on assignment version | No duplicate alert storm |

**V1 dispatch**

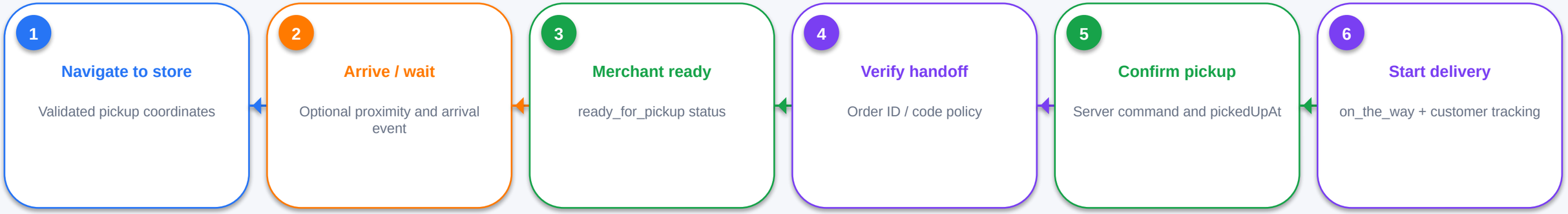
Manual admin assignment is acceptable for launch. Design the command and data model so automated dispatch can be added later without changing canonical order ownership.

PICKUP WORKFLOW

# Navigate to Store, Wait and Confirm Pickup

Pickup begins only for the assigned driver and an active paid order.

## 1 Pickup sequence




## 2 Pickup state/action matrix

| Order state | Driver UI | Allowed command | Cross-app result |
|---|---|---|---|
| **driver_assigned + confirmed** | Navigate / wait | Optional arrivedAt | Admin/merchant see arrival |
| **driver_assigned + preparing** | Wait / contact support | Issue only | Customer sees preparing |
| **ready_for_pickup** | Verify and confirm pickup | driverConfirmPickup | Status becomes on_the_way |
| **cancelled** | Stop route | Acknowledge only | Remove from active queue |
| **assignment removed** | Refresh / stop handling | No pickup command | New driver receives assignment |
| **already on_the_way** | Resume delivery screen | Idempotent read | No duplicate transition |

**Handoff integrity**

A pickup code, QR or merchant confirmation can be added later. Regardless of method, the backend must verify order, assigned driver and current state before writing pickedUpAt/on_the_way.

# Route Navigation, Location Sharing and Freshness

Location data is operational, time-bound and normalized for customer/admin consumption.

## 1 Navigation and route data

### 1 Pickup route

Driver -> store route using validated store coordinates.

### 2 Delivery route

Store/current position -> customer address snapshot.

### 3 External navigation

Open approved maps app using coordinates; retain order state in Driver App.

### 4 Route metadata

distance, ETA, route source and calculatedAt normalized by service.

### 5 Failure

Show address and retry; never invent coordinates or ETA.

## 2 Location update contract

| Field / policy | Requirement | Consumer |
|---|---|---|
| driverId / orderId | Authenticated assigned driver only | Rules / Function |
| lat / lng | Valid finite coordinates and accuracy metadata | Admin / customer projection |
| recordedAt | Server-normalized or bounded client timestamp | Freshness logic |
| accuracy / heading / speed | Optional, privacy-reviewed | Routing / operations |
| update cadence | Throttled by movement/time and app state | Battery + cost control |
| retention | Active delivery plus approved history window | Privacy policy |
| stale threshold | Creates DRIVER_LOCATION_STALE activity | Admin needs-action |

**Privacy and product truth**

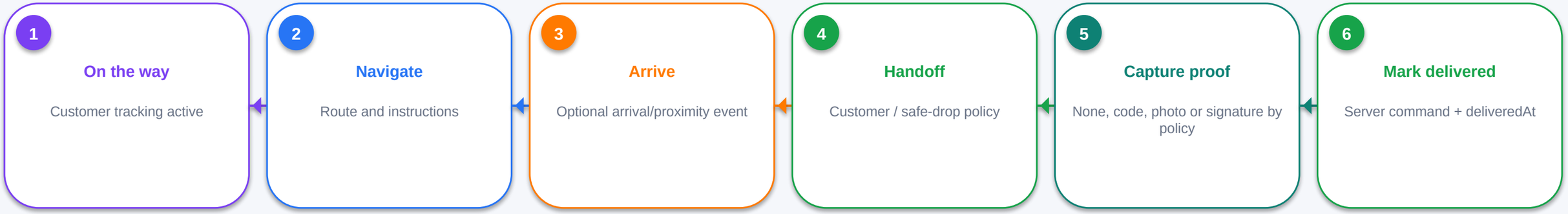
Location sharing must be visibly enabled for active delivery operations. Customer-facing tracking should use a scoped projection and stop when the order is completed or cancelled.

DELIVERY WORKFLOW

# On-the-Way, Arrival and Delivery Confirmation

Delivery completion requires the assigned driver, valid current state and the approved proof policy.

## 1 Delivery sequence




## 2 Proof and completion matrix

| Policy element | V1 recommendation | Data treatment |
|---|---|---|
| **Recipient confirmation** | Optional simple confirmation / delivery note | Do not collect unnecessary identity data |
| **Delivery code** | Strong proof when implemented | Hashed/short-lived verification |
| **Photo proof** | Only if policy and consent support it | Private Storage, scoped access, retention limit |
| **Signature** | Later phase if operationally justified | Sensitive data controls |
| **Safe drop** | Explicit customer instruction and driver note | Snapshot into delivery result |
| **Delivered command** | Always required | Valid assignment + on_the_way + idempotency |
| **Completion result** | Canonical delivered order | Stop location sharing and notify all channels |

**No false completion**

A local success screen is not proof that delivery status persisted. Show completion only after the backend returns the refreshed canonical order.

EXCEPTIONS & SAFETY

# Driver Issues, Escalation and Contact Boundaries

Structured issue reporting creates actionable admin work without allowing unsafe status manipulation.

## 1 Issue categories

### 1 ADDRESS_PROBLEM

Invalid pin, inaccessible address or missing detail.

### 2 CUSTOMER_UNREACHABLE

Record attempts and wait policy, not private notes.

### 3 MERCHANT_DELAY

Order not ready beyond threshold.

### 4 VEHICLE / SAFETY

Breakdown, accident, unsafe location or emergency.

## 2 Driver actions

### 1 Report issue

Category, safe note, related order and optional evidence.

### 2 Contact support

In-app or masked channel; avoid exposing personal numbers.

### 3 Request reassignment

Backend/admin decision; driver cannot self-clear assignment.

### 4 Pause location appropriately

Safety/privacy policy controls behavior, not hidden client state.

## 3 Admin/system response

### 1 Activity queue

Priority, owner, status and related entities.

### 2 Reassign / cancel

Validated privileged command with reason.

### 3 Customer notification

Safe delay or resolution message.

### 4 Audit

Actor, previous/new state and timestamps.

# Push, Resume and Background Operation

Background behavior is explicit, platform-aware and tested against battery and permission limits.

## 1 Push events

### 1 New assignment
Deep link to assigned order and current version.

### 2 Merchant ready
Pickup-ready change for assigned order.

### 3 Admin reassignment / cancellation
Immediate active-screen invalidation.

### 4 Support update
Issue resolution or required action.

## 2 Resume behavior

### 1 Reauthenticate
Confirm valid session and active driver status.

### 2 Refresh assignments
Never rely only on stale local queue.

### 3 Reconcile commands
Resolve pending command IDs and canonical result.

### 4 Restart tracking
Only when an active eligible delivery exists.

## 3 Background constraints

### 1 OS permissions
Explain foreground/background location separately.

### 2 Throttling
Movement/time thresholds and lifecycle-aware cadence.

### 3 No guaranteed execution
Design for delayed/terminated app states.

### 4 Server freshness monitor
Scheduled/triggered logic detects stale location independently.

# Driver Read/Write Contract

Driver data access is assignment-scoped and commands are narrow.

## 1 Readable projections

| Data | Scope | Use |
|---|---|---|
| users/{uid} | self | profile, status, availability |
| orders/{orderId} | driverId == uid or eligible projection | assigned work and lifecycle |
| driverAssignments | self/current | assignment version and acknowledgement |
| notifications/{id} | recipientUid == uid | alerts and read state |
| driverLocations/current | self write / scoped read | active location freshness |
| activities/{id} | own submitted / permitted status | issue tracking |
| stores snapshot / pickup projection | assigned order only | pickup details |

## 2 Driver commands

### 1 setDriverAvailability

Validates active status and writes online/offline with server timestamp.

### 2 acceptAssignment

Checks assignment version and ownership atomically.

### 3 confirmPickup

Checks driverId and ready/allowed state.

### 4 updateDriverLocation

Validates active assignment and bounded payload.

### 5 markDelivered

Checks on_the_way, proof policy and idempotency.

### 6 reportDriverIssue

Creates activity; does not silently change order state.

**Protected fields**

Driver App cannot change order totals, payment fields, store/item snapshots, customerId, merchant state or assignment ownership through direct Firestore writes.

SECURITY & PRIVACY

# Driver Authorization, Location Protection and Abuse Tests

The driver role is high-impact because it can affect physical fulfillment and customer location visibility.

## 1 Authorization layers

### 1 Route guard

Authenticated active driver required.

### 2 Rules

Assigned-order ownership and protected fields.

### 3 Functions

Assignment/status/location commands.

### 4 App Check

Client attestation in addition to Auth/rules.

## 2 Location privacy

### 1 Purpose limit

Active-delivery operations only.

### 2 Scoped customer view

No unrestricted driver location collection reads.

### 3 Retention

Clear approved window and cleanup.

### 4 Consent / controls

Visible state and OS permission handling.

## 3 Abuse scenarios

### 1 Claim another order

Denied by transaction/version rules.

### 2 Fake status jump

Delivered from assigned/ready denied.

### 3 Spoofed/stale location

Accuracy/time/anomaly monitoring and audit.

### 4 Suspended token

Operational reads/writes fail closed.

CODEBASE ARCHITECTURE

# Driver App Monorepo Boundaries

Driver App remains an Expo React Native app and imports platform-neutral domain logic.

## 1 Driver application

### 1 routes/
workspace, order, notifications, account

### 2 components/
delivery cards, stepper, map panels

### 3 adapters/
location, notifications, deep links

### 4 state/
active order and UI state

### 5 storage/
secure/local operational cache

## 2 Shared packages

### 1 types/
DriverOrder and Location

### 2 schemas/
driver commands

### 3 services/
orders, assignments, notifications

### 4 maps/
route and coordinate contracts

### 5 utils/
freshness and status helpers

## 3 Firebase backend

### 1 Functions
assignment and status commands

### 2 Rules
driver ownership

### 3 Indexes
driverId/status queries

### 4 Messaging
assignment events

### 5 Storage
proof media if enabled

## 4 Native boundary

### 1 Expo config
preserve EAS project ID

### 2 Permissions
location/notifications

### 3 Background tasks
platform adapter

### 4 Maps SDK
driver-only UI adapter

### 5 No web UI import
strict dependency direction

**Migration constraint**

Preserve the existing Driver App EAS configuration and Firebase project. Move code incrementally and verify sign-in, assigned orders, location and status updates before deleting any old path.

ENGINEERING QUALITY

# Battery, Reliability, Observability and Tests

Driver operations must remain recoverable under weak connectivity and mobile lifecycle interruptions.

## 1 Performance / battery

### 1 Adaptive location cadence

Update by time/movement/state, not every render.

### 2 Map lifecycle

Render only active route and stop when hidden.

### 3 Bounded listeners

Assigned/active work only; unsubscribe after completion.

### 4 Image restraint

Avoid heavy admin-style media/table UIs.

## 2 Reliability

### 1 Idempotent status commands

Stable command ID for pickup/delivered retries.

### 2 Conflict refresh

Stale assignment returns current canonical state.

### 3 Offline queue policy

Location may buffer under policy; critical status waits for server confirmation.

### 4 Recovery

Resume resolves active assignment and pending commands.

## 3 Observability / tests

### 1 Metrics

Assignment acceptance, pickup wait, route, stale location and delivery time.

### 2 Logs

Command ID, driver/order, duration and result.

### 3 Mobile smoke

Permissions, background/resume, route and completion.

### 4 Emulator security

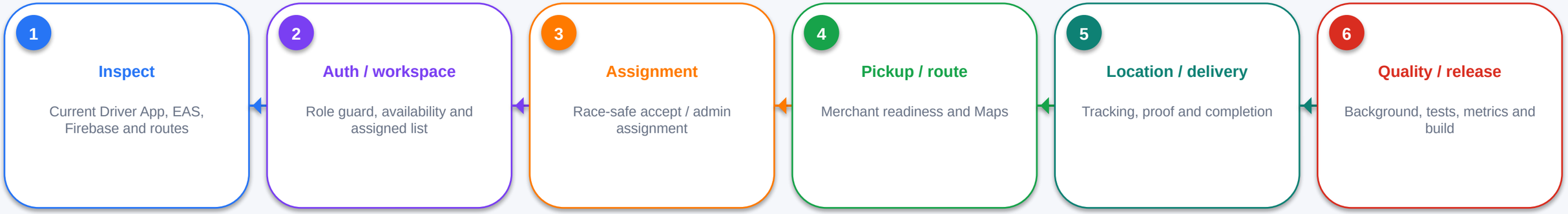
Cross-driver denial and transition matrix.

IMPLEMENTATION OPERATING MODEL

# Driver App Development Roadmap

Build from secure identity and assignment toward route/location and robust completion.

## 1 Phased implementation




## 2 Phase exit criteria

| Phase | Exit criteria | Critical failure to prevent |
|---|---|---|
| **1. Discovery** | Exact code/data/deployment paths documented | Second Firebase/EAS project |
| **2. Identity/workspace** | Only active driver sees scoped work | Cross-driver reads |
| **3. Assignment** | One driver and version-safe acknowledgement | Race overwrite |
| **4. Pickup/maps** | Ready/pickup transitions and route fallback work | Driver bypasses merchant state |
| **5. Location/delivery** | Privacy-scoped tracking and idempotent delivered | False/stale completion |
| **6. Release** | Battery, resume, security and builds verified | Unbounded background/listener behavior |

**Reusable Codex instruction**

Resume current progress. Inspect driver-app, shared packages, Functions, rules and EAS configuration. Implement one bounded workflow, preserve one backend and current IDs, run emulator/mobile checks and report all changed files and deployment effects.

REFERENCE CONTROL

# Driver App Traceability and Decisions

Traceability, current decisions and implementation assumptions.

## 1 Source-to-module traceability

| Source | Captured architecture detail | Used in this document |
|---|---|---|
| Admin blueprint - system context | Driver accepts, navigates, shares location and delivers | Pages 2-3 |
| Current deployed driver experience | Assigned orders, notifications, detail, map, location toggle and statuses | Pages 3, 5, 7-11 |
| Admin blueprint - order lifecycle | Driver assignment, on-the-way and delivered ownership | Pages 6-9 |
| Admin blueprint - needs action | No driver, delay and stale location reason codes | Pages 8, 10, 15 |
| Admin blueprint - security | Assigned-order scope, Functions and audit | Pages 4, 12-16 |
| Google Maps platform | Routes, coordinates and ETA enrichment | Pages 7-9 |
| Monorepo plan | Expo Driver App with shared domain packages and preserved EAS ID | Pages 14-16 |

## 2 Normalized decisions

### Dispatch v1

Manual admin assignment is acceptable; design for future automation.

### Location

Purpose-limited, throttled, freshness-tracked and removed from customer view after completion.

### Status truth

Pickup/on-the-way/delivered are server-validated commands.

### Proof

Start simple; add code/photo/signature only with policy and privacy controls.

### Mobile stack

Expo/React Native remains native; no admin-web component reuse.

### One backend

Existing Firebase project and canonical orders are preserved.

### Source precedence

Current repo/deployment > approved architecture > mockup intent.

### Document status

Architecture synthesis for implementation. Before changing code, validate exact collection names, existing Cloud Functions, deployed rules, current routes and environment variables against the monorepo.